

TP0 – Herramientas

Objetivos

Completar cuestionarios: [Encuesta de Bienvenida](#) y [Conformación de Grupos](#)

Registrar, Descargar, Instalar, Licenciar y Usar Herramientas

Usar [STM32CubeIDE](#)

Generar proyecto [STM32](#) con [FreeRTOS](#)

Editar/compilar/depurar programas en [C](#) con [FreeRTOS](#)

Configurar el [IDE](#) para depurar en modo [semihosting](#)

Adoptar metodología orientada a la portabilidad/reúso de [código](#)

Identificar [Patrones de Diseño de Software](#)

Usar [Git](#) y [GitHub](#)

Generar/actualizar [repositorio](#) por línea de [comando](#)

Repasar programación en [C](#)

Herramientas

[Integrated Development Environment \(IDE\) - Workspace – Project](#)

[Version Control System \(VCS\) – Repository](#)

Programación en C de Sistemas Embebidos

[Semihosting \(printf/scanf\)](#)

[Multitasking - Event-Triggered Systems \(ETS\) and Time-Triggered \(TTS\)](#)

[Embedded C Coding Standard - Repaso de programación en C](#)

Actividades

1er encuentro

[Encuesta de Bienvenida](#) ([Tarea para el hogar](#))

[Conformación de Grupos](#) ([Tarea para el hogar](#))

[TP0 – Actividad 00 – Herramientas](#) ([Tarea para el hogar](#))

[TP0 – Actividad 01 – 1er Proyecto p/placa NUCLEO-F103RB con FreeRTOS \(STM32 Project - Led Blink\)](#)

2do encuentro

[TP0 – Actividad 02 – 2do Proyecto p/placa NUCLEO-F103RB con FreeRTOS \(Semihosting "Hello World\n!"\)](#)

[TP0 – Actividad 03 – 3er Proyecto p/placa NUCLEO-F446RE con FreeRTOS \(Application\)](#)

[TP0 – Actividad 04 - Repaso de programación en C](#) ([Tarea para el hogar](#))

Resolución en grupos de **3** alumnos, requiere **presentación** por parte de un **responsable** del grupo

TP0 – Actividad 00 – Herramientas

Paso 01: Completar los cuestionarios: **Encuesta de Bienvenida** (**Campus FIUBA Posgrado**) y **Conformación de Grupos** (**Campus FIUBA Posgrado**).

Paso 02: Registrarse en el sitio de **GitHub** (<https://github.com/>).

Paso 03: Registrarse en el sitio de **STMicroelectronics** (https://www.st.com/content/st_com/en.html).

Paso 04: Registrarse en el sitio de **Itemis Create Cloud Editor** (<https://create.itemis.io/login>).

Herramienta vía **navegador web**, funciona correctamente con la **versión actualizada** de **Chrome**.

Paso 05: Descargar, instalar y licenciar (Academic - € 0 - con email del dominio FIUBA => @fi.uba.ar) en su Computadora Personal **Itemis Create Desktop** (<https://www.itemis.com/en/products/itemis-create/>,
<https://info.itemis.com/products/itemis-create/download/&> <https://www.itemis.com/en/products/itemis-create/licenses/>).

Paso 06: Descargar e instalar en su Computadora Personal **Git** (<https://git-scm.com/downloads>).

Instalación **Git Windows** y creación de **repositorio** p/proyecto **STM32** (<https://www.youtube.com/watch?v=eyDKt-ppAeA>).

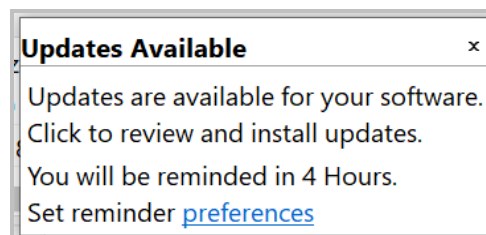
Instalación **Git / GitHub Linux** (https://github.com/cpatagon/RTOSI_github).

Paso 07: Descargar e instalar en su Computadora Personal **STM32CubeIDE** versión **1.19.0**.
(<https://www.st.com/en/development-tools/stm32cubeide.html>).

Instalación **STM32CubeIDE 1 19 0 Linux** (<https://www.youtube.com/watch?v=bLY94ls9TSQ>). Puede ser que ya no sea necesario descomprimir el archivo por lo que habría que saltar el paso de la descompresión que aparece en el video.

Instalación **STM32CubeIDE 1.19.0 MacOS** (<https://www.youtube.com/watch?v=6DE2fUHKv9g>).

Si **STM32CubeIDE** avisa que hay **actualizaciones** disponibles (ver imagen al pie), **no instalarlas**.



Paso 08: Descargar e instalar en su Computadora Personal **STM32CubeProgrammer**
(<https://www.st.com/en/development-tools/stm32cubeprog.html>).

TP0 – Actividad 01 – 1er Proyecto p/placa NUCLEO-F446RE con FreeRTOS

Siguiendo los pasos de la guía **STM32CubeIDE Basic project on FreeRTOS**, generar su primer **STM32 Project** con **FreeRTOS**, para su placa **NUCLEO-F446RE**.

Buscar en **main.c** el siguiente comentario: `/* USER CODE BEGIN 5 */`. A continuación está la función **StartDefaultTask()**, dentro del cuerpo del **for (;)**. Copiar y Pegar lo siguiente:

```
HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);  
osDelay(1000);
```

Compilar/depurar el programa (comprobar que LD2 titila).

Cerrar la sesión de depuración.

Crear repositorio y subir el **STM32 Project** al mismo.

Escribir en **Agregar entrega** (Texto en línea) el **link** del repositorio creado en **GitHub**:

Campus FIUBA Posgrado => 9. Sistemas Operativos de Tiempo Real (I) > Evaluaciones > TP0 - Herramientas

Escribir en **Agregar entrega** (Texto en línea): URL del repositorio.

Referencias:

- **Embedded C Coding Standard**, link: <https://barrgroup.com/free-downloads>
 - **C Style Guide - Embedded C Coding Standard**, link: https://barrgroup.com/sites/default/files/barr_c_coding_standard_2018.pdf
- Algunas de estas cosas ya las hizo y están documentadas en: **TA134 - TdSE - Guia TP0 – Herramientas**, link: https://drive.google.com/file/d/1kPUGiVN_nOzFDuufB2XQDArIbdoTkIN5/view?usp=drive_link
- **STM32CubeIDE Basic project on FreeRTOS**, link: https://drive.google.com/file/d/1L6k-XsB0RudhDhPQpX-K_Mxdutz321N/view?usp=drive_link
- **UM2609 - User manual - STM32CubeIDE User Guide**, link: https://www.st.com/resource/en/user_manual/um2609-stm32cubeide-user-guide-stmicroelectronics.pdf
 - **How to use semihosting with STM32CubeIDE and STM32**, link: <https://community.st.com/t5/stm32-mcus/how-to-use-semihosting-with-stm32cubeide-and-stm32/ta-p/49742>
- **AN4989 STM32 - Microcontroller Debug Toolbox**, link: https://www.st.com/resource/en/application_note/an4989-stm32-microcontroller-debug-toolbox-stmicroelectronics.pdf
 - **How to enable FreeRTOS™ Run Time and Stack Usage view**, link: <https://community.st.com/t5/stm32-mcus/how-to-enable-freertos-run-time-and-stack-usage-view/ta-p/627524>
- Workspace Name: **sotri_workspace**
- Project Name: **sotri-tp0_01-stm32_project**
- Repository name: **sotri-tp0_01-herramientas**
 - Description: **CESE – Sistemas Operativos de Tiempo Real I – Trabajo Práctico N°: 0 – Herramientas**

TP0 – Actividad 02 – 2do Proyecto p/placa NUCLEO-F446RE con FreeRTOS

Para **generar** un nuevo **proyecto STM32**: **copiar** el archivo **sotri-tp0_01-stm32_project** y **pegar/renombrar** como **sotri-tp0_02-semihosting**.

Renombrar el archivo **sotri-tp0_01-stm32_project.ioc** como **sotri-tp0_02-semihosting.ioc**.

Renombrar el archivo **sotri-tp0_01-stm32_project.cfg** como **sotri-tp0_02-semihosting.cfg**.

Borrar archivos/carpetas de compilación/programación/depuración **sotri-tp0_01-stm32_project.launch** y **Debug**.

Buscar en **main.c** el siguiente comentario: **/* USER CODE BEGIN Includes */**. **Copiar** y **Pegar** lo siguiente **a continuación**:

```
#include "stdio.h"
```

Buscar en **main.c** el siguiente comentario: **/* USER CODE BEGIN 0 */**. **Copiar** y **Pegar** lo siguiente **a continuación**:

```
extern void initialise_monitor_handles(void);
```

Buscar en **main.c** el siguiente comentario: **/* USER CODE BEGIN 1 */**. **Copiar** y **Pegar** lo siguiente **a continuación**:

```
initialise_monitor_handles();
```

Buscar en **main.c** el siguiente comentario: **/* USER CODE BEGIN 5 */**. A continuación está la función **StartDefaultTask()**, dentro del cuerpo del **for (;)**. **Copiar** y **Pegar** lo siguiente:

```
printf("Hello World!\n");
```

Compilar el programa.

Siguiendo los pasos de la guía: **AN4989 STM32 - Microcontroller Debug Toolbox** (=> "7. Printf debugging => 7.4. Semihosting => 7.4.3. STM32CubeIDE"), configure las **propiedades** del proyecto y el **debugger** para trabajar en modo **semihosting** con un **STM32 Project** con **FreeRTOS**.

Depurar el programa (**comprobar** que por consola sale el mensaje **"Hello, World!\n"**).

Cerrar la sesión de depuración.

Subir el **STM32 Project** al **repositorio**.

Referencias:

- **Embedded C Coding Standard**, link: <https://barrgroup.com/free-downloads>
 - **C Style Guide - Embedded C Coding Standard**, link: https://barrgroup.com/sites/default/files/barr_c_coding_standard_2018.pdf
- Algunas de estas cosas ya las hizo y están documentadas en: **TA134 - TdSE - Guia TP0 – Herramientas**, link: https://drive.google.com/file/d/1kPUGiVN_nOzFDuufB2XQDArIbdoTkIN5/view?usp=drive_link
- **STM32CubeIDE Basic project on FreeRTOS**, link: https://drive.google.com/file/d/1L6k-XsB0RudhDhPQpX-K_Mxdutz321N/view?usp=drive_link

- **UM2609 - User manual - STM32CubeIDE User Guide**, link:
https://www.st.com/resource/en/user_manual/um2609-stm32cubeide-user-guide-stmicroelectronics.pdf
 -
 - **How to enable FreeRTOS™ Run Time and Stack Usage view**, link:
<https://community.st.com/t5/stm32-mcus/how-to-enable-freertos-run-time-and-stack-usage-view/ta-p/627524>
- **AN4989 STM32 - Microcontroller Debug Toolbox**, link:
https://www.st.com/resource/en/application_note/an4989-stm32-microcontroller-debug-toolbox-stmicroelectronics.pdf
 - **How to use semihosting with STM32CubeIDE and STM32**, link:
<https://community.st.com/t5/stm32-mcus/how-to-use-semihosting-with-stm32cubeide-and-stm32/ta-p/49742>
- Workspace Name: **sotri_workspace**
- Project Name: **sotri-tp0_02-semihosting**
- Repository name: **sotri-tp0_01-herramientas**
 - Description: **CESE – Sistemas Operativos de Tiempo Real I – Trabajo Práctico N°: 0 – Herramientas**

TP0 – Actividad 03 – 3er Proyecto p/placa NUCLEO-F446RE con FreeRTOS

Para **generar** un nuevo **proyecto STM32**: **copiar** el archivo **sotri-tp0_02-semihosting** y **pegar/renombrar** como **sotri-tp0_03-application**.

Renombrar el archivo **sotri-tp0_02-semihosting.ioc** como **sotri-tp0_03-application.ioc**.

Renombrar el archivo **sotri-tp0_02-semihosting.cfg** como **sotri-tp0_03-application.cfg**.

Borrar archivos/carpetas de compilación/programación/depuración **sotri-tp0_02-semihosting.launch** y **Debug**.

Generar el archivo de depuración **sotri-tp0_03-application.launch**.

Compilar/Depurar el programa (**comprobar** que por consola sale el mensaje "Hello, World!\n").

Descargar **app.zip**, **extraer** la carpeta: **app** de **app.zip**, **Copiarla** y **Pegarla** en la raíz de su **STM32 Project** con **FreeRTOS**.

Modificar las **propiedades** del proyecto para incluir/compilar los fuentes de la carpeta **app**.

Buscar en **main.c** el siguiente comentario: **/* USER CODE BEGIN Includes */**. **Copiar** y **Pegar** lo siguiente **a continuación**:

```
/* Demo includes. */
#include "logger.h"
#include "dwt.h"

/* Application includes. */
#include "app.h"
```

Buscar en **main.c** el siguiente comentario: **/* USER CODE BEGIN 0 */**. **Copiar** y **Pegar** lo siguiente **a continuación**:

```
#if (1 == LOGGER_CONFIG_USE_SEMIHOSTING)
extern void initialise_monitor_handles(void);
#endif
```

Buscar en **main.c** el siguiente comentario: **/* USER CODE BEGIN 1 */**. **Copiar** y **Pegar** lo siguiente **a continuación**:

```
#if (1 == LOGGER_CONFIG_USE_SEMIHOSTING)
initialise_monitor_handles();
#endif
```

Buscar en **main.c** el siguiente comentario: **/* USER CODE BEGIN 2 */**. **Copiar** y **Pegar** lo siguiente **a continuación**:

```
/* add application, ... */
app_init();
```

Buscar en **main.c** el siguiente comentario: **/* USER CODE BEGIN RTOS_QUEUES */ /* add queues, ... */**. **Copiar** y **Pegar** lo siguiente **a continuación**:

```
#ifdef _defaultTask_
```

Buscar en [main.c](#) el siguiente comentario: `/* USER CODE BEGIN RTOS_THREADS */`. Copiar y Pegar lo siguiente a continuación:

```
#endif
```

Compilar/depurar el programa (comprobar que se ejecutan varias tareas).

Siguiendo los pasos de la guía: **UM2609 - User manual - STM32CubeIDE User Guide** (=> **3.4 RTOS-aware debugging => 3.4.2 FreeRTOS™**), modifique el proyecto y el **debugger** para habilitar la vista de uso de stack y tiempo de ejecución de un **STM32 Project** con **FreeRTOS**.

Compilar/depurar el programa (comprobar que se ejecutan varias tareas).

Cerrar la sesión de depuración.

Subir el **STM32 Project** al [repositorio](#).

Referencias:

- **Embedded C Coding Standard**, link: <https://barrgroup.com/free-downloads>
 - **C Style Guide - Embedded C Coding Standard**, link: https://barrgroup.com/sites/default/files/barr_c_coding_standard_2018.pdf
- Algunas de estas cosas ya las hizo y están documentadas en: **TA134 - TdSE - Guia TP0 – Herramientas**, link: https://drive.google.com/file/d/1kPUGiVN_nOzFDuufB2XQDArIbdoTkIN5/view?usp=drive_link
- **STM32CubeIDE Basic project on FreeRTOS**, link: https://drive.google.com/file/d/1L6k-XsB0RudhDhPQpX-_K_Mxdutz321N/view?usp=drive_link
- **UM2609 - User manual - STM32CubeIDE User Guide**, link: https://www.st.com/resource/en/user_manual/um2609-stm32cubeide-user-guide-stmicroelectronics.pdf
 - **How to enable FreeRTOS™ Run Time and Stack Usage view**, link: <https://community.st.com/t5/stm32-mcus/how-to-enable-freertos-run-time-and-stack-usage-view/ta-p/627524>
- **AN4989 STM32 - Microcontroller Debug Toolbox**, link: https://www.st.com/resource/en/application_note/an4989-stm32-microcontroller-debug-toolbox-stmicroelectronics.pdf
 - **How to use semihosting with STM32CubeIDE and STM32**, link: <https://community.st.com/t5/stm32-mcus/how-to-use-semihosting-with-stm32cubeide-and-stm32/ta-p/49742>
- Workspace Name: **sotri_workspace**
- Project Name: **sotri-tp0_03-application**
- Repository name: **sotri-tp0_01-herramientas**
 - Description: **CESE – Sistemas Operativos de Tiempo Real I – Trabajo Práctico N°: 0 – Herramientas**

TP0 – Actividad 04 – Repaso de Programación en C

Paso 01: Dados los tres lados de un triángulo, determine si es Equilátero, Isósceles o Escaleno.

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)

Paso 02: Imprima por pantalla los primeros 10 múltiplos del 3.

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)

Paso 03: Escriba una función que cuente la cantidad de bits en 1 de un entero dado.

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)

Paso 04: Escriba una función que muestre el estado de los 8 bits de un byte.

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)

Paso 05: Escriba una función que ponga en uno un bit dado de un entero dado, sin modificar los demás.

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)

Paso 06A: Escriba una función que muestre el estado de los 8 bits de un byte.

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)

Paso 06B: Escriba una función que ponga en uno un bit dado de un entero dado, sin modificar los demás.

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)

Paso 06C: Haga una librería estática con las funciones de combinatoria y enlázela con un programa que la use.

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)

Paso 07: Diseñe e implemente una cola circular de 10 enteros. Las operaciones que se hacen sobre la cola son: **Inicializar**, **poner** un dato, **sacar** un dato, **obtener** la cantidad de datos en la cola, **obtener** el espacio libre, **verificar** si está llena, **verificar** si está vacía mostrar el contenido.

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)

Paso 08: Diseñe e implemente las funciones necesarias para leer los datos de un archivo.

El archivo contendrá mensajes.

Cada mensaje está formado por un encabezado, un dato y un fin de mensaje.

El encabezado del mensaje se define como la secuencia: < identificador >

El fin de mensaje se define como: < / identificador >

El dato de un mensaje puede ser un número del 1 al 255, que ocupa un byte, o también otro mensaje.

Ejemplo de la estructura de un mensaje en el archivo:

...Mensaje anterior... <msg> DATO </msg> ...Mensaje siguiente...

Ejemplo de contenido del archivo

<trama><medicion>31</medicion><medicion>31</medicion></trama>...

...<fecha><dia>5</dia><mes>2</mes><año>14</año></fecha>

En forma de árbol:

```
<trama>
  <medicion> 31 </medicion>
  <medicion> 31 </medicion>
</trama>
<fecha>
  <dia> 5 </dia>
  <mes> 2 </mes>
  <año> 14 </año>
</fecha>
```

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)

Paso 09A Describa que funciones y que variables debe contener una librería genérica que maneje GPIOs de un microcontrolador. ¿Cómo las implementaría en C?

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)

Paso 09B: Describa que funciones y que variables debe contener una librería genérica que maneje un BUS de datos en un microcontrolador. ¿Cómo las implementaría en C?

Este ejercicio es para hacer con el compilador en línea OnlineGDBbeta (https://www.onlinegdb.com/online_c_compiler)
